Automated build and ANT practical

Mike Jackson

12 July 2016

1 Introduction

ANT (Another Neat Tool) (http://ant.apache.org/) is a popular open source automated build tool written in Java. ANT can be used for many of the same tasks that Make is, however, it includes a number of powerful features specifically of use for Java developers.

A number of integrated development environments (e.g. Eclipse, JBuilder and NetBeans) and other tools (e.g. Emacs) include built-in support for writing ANT build files and running ANT.

This introduction assumes you are familiar with Make concepts such as targets, dependencies, actions, rules, and Makefiles.

You will need to download some code for these practicals. This code is available through the LEARN area for this course. To access these files:

1. Log-in to LEARN at http://www.learn.ed.ac.uk OR via http://www.myed.ed.ac.uk, using your EASE username and password.
2. Click My Courses.
3. Click Programming Skills(2016-2017)[SS1-SEM1].
4. Click Labs.
5. Under Automated build, click AutomatedBuild.
6. Right-click on build-ant.zip and select Save As…

Copy the code to your desktop or `ph-cplab.ph.ed.ac.uk` and decompress it. This should produce a directory called `build-ant`.

2 A first ANT build file

Make has Makefiles which are plain-text files. ANT, in contrast, has build files, which are written in XML. As a result they are more verbose than Makefiles, but they do not have Make's cryptic macro syntax, nor do they rely on the use of tab characters.

Here is a simple “hello world” Makefile:

```
whoiam:
```

```
    @echo "I am a Makefile"



hello: whoiam

    @echo "Hello world!"
```

Here is the equivalent ANT build file:

```
<?xml version="1.0"?>

<project name="HelloWorld" default="hello" basedir=".">

    <target name="whoiam">

        <echo message="I am an ANT file!"/>

    </target>

   <target name="hello" depends="whoiam">

       <echo message="Hello, World!"/>

    </target>

</project>
```

The first line says that this file is an XML document.

The `project` element defines some meta-information about this build file, including:

- `name`: a name.
- `default`: default target, the target that is run if the user does not explicitly select one. In Make, the default target is usually the first one in the Makefile.
- `basedir`: base directory. Any relative paths in the file will be considered to be relative to this directory. Here it is set to the current directory, `.`.

There are then two targets. ANT targets are equivalent to rules in Make. Targets are defined within the project element, using the `target` element.

The first target, `whoiam`, has one action, which ANT calls a task. The `echo` task prints a message. All ANT tasks are implemented in Java. For a list of ANT tasks, see https://ant.apache.org/manual/tasksoverview.html.

The second target `hello`, also prints a message. It also has a dependency on `whoiam`. Like Make, ANT targets can have zero or more dependencies. These are provided as a comma separated list e.g.

```
<target name="A" depends="B,C,D">

    ...tasks...
```

Dependencies run from left to right and are executed in this order. Here, when `A` is invoked, the targets `B`, `C` and `D` will be run in order and, only then, the tasks for `A` will be run.

3 Run ANT

If we put the Makefile in the previous section into a file called `Makefile`, and run Make, we get:

```
$ make

I am a Makefile

Hello world!
```

Similarly, we can put the build file in the previous section into a file called `build.xml`, and then run ANT as follows:

```
$ ant
```

It will print out what it is doing:

```
whoiam:

     [echo] I am an ANT file!

hello:

     [echo] Hello, World!

BUILD SUCCESSFUL

Total time: 0 seconds
```

In the same way that Make looks for a file called `Makefile`, ANT looks for a file called `build.xml`. If our file was called `build-hello.xml` we would need to tell ANT the name of this file, as follows:

```
$ ant -f hello-build.xml
```

Like Make, we can tell ANT what target to run:

```
$ ant whoiam

Buildfile: build.xml

whoiam:

     [echo] I am an ANT file!

BUILD SUCCESSFUL

Total time: 0 seconds
```

We can see a list of available targets, and other information about the ANT file:

```
$ ant -p
```

```
Buildfile: build.xml

Main targets:

Other targets:

 hello

 whoiam

Default target: hello
```

4 Properties

In Make we can define names for values, called macros. In ANT, the equivalent of macros are called properties. We can define a property to hold our name as follows:

```
<property name="your.name" value="Mike"/>
```

Properties can be defined within specific targets or within the overall `project` element, in which case their value is available to all the targets in the build file.

Let's edit `hello` target to define this property and use it in its message:

```
<target name="hello" depends="whoiam">

    <property name="your.name" value="Mike"/>

    <echo message="Hello, ${your.name}!"/>

</target>
```

Properties are dereferenced using the syntax `${NAME}` e.g. `${your.name}`.

Running ANT now gives:

```
$ ant

Buildfile: build.xml

whoiam:

     [echo] I am an ANT file!

hello:

     [echo] Hello, Mike!

BUILD SUCCESSFUL

Total time: 0 seconds
```

Values of properties are resolved at run-time and once set, a property's value cannot be updated. If we try and redefine the value of `your.name`:

```
<property name="your.name" value="Mike"/>

<property name="your.name" value="Fred"/>
```

Running ANT still gives:

```
$ ant

...

     [echo] Hello, Mike!

...
```

5 Compile Java

Assume we have a directory `src` containing `src/dna/Sequence.java` which defines the Java implementation of a class, `dna.Sequence`, that represents DNA sequences. When compiled, this class can be run from the command-line and calculate the molecular weight of a given sequence e.g.

```
$ java dna.Sequence GATTACCA

1909.6000000000001
```

`test/dna/SequenceTest.java` contains a unit test class `dna.SequenceTest`, written using the JUnit (http://junit.org) test framework.

Let's write a new `build.xml` file to compile and run this Java code. First write a skeleton `build.xml` file:

```
<?xml version="1.0"?>

<project name="dna" default="all" basedir=".">



</project>
```

Now let's define a target to create some directories to hold our compiled classes and an ANT JAR file:

```
<target name="init">

    <mkdir dir="build"/>

    <mkdir dir="build/lib"/>

    <mkdir dir="build/classes"/>

</target>
```

Let's test our ANT script so far:

```
$ ant init

$ ls build
```

Now, add a target to remove the build directory:

```
<target name="clean">

    <delete dir="build"/>

</target>
```

Now let's provide a target to compile our source code, in `src` and put the compiled code in `build/classes`:

```
<target name="compile" depends="init">

    <javac srcdir="src" destdir="build/classes">

        <include name="**/*.java"/>

    </javac>

</target>
```

Our target depends on `init` so the directories for the compiled code will always be created first, if they do not exist. We then use the `javac` task to compile our code, specifying:

- `src`: the directory in which our source code is located.
- `destdir`: the directory in which our compiled code is to be put.
- `include`: the files to compile. `**/*.java` means "all files which have the suffix `.java` and are in any descendent directory of the directory specified in `srcdir`".

If we run:

```
$ ant compile
```

we can compile the code. If there is already compiled code in the `destdir` then only those files with no `.class` file or whose `.class` file is older than the corresponding `.java` file are recompiled.

Let's now add another target to take the compiled code and bundle it into a Java Archive, or JAR file, using the `jar` task:

```
<target name="jar" depends="compile">

    <jar destfile="build/lib/dna.jar"

         basedir="build/classes"/>

</target>
```

We specify:

- `destfile`: the location of the JAR file to be created.
- `basedir`: the location of the compiled code to be bundled.

Let's clean up and rebuild everything:

```
$ ant clean

$ ant jar

$ ls build/lib/
```

6 Run Java

We can run Java via ANT using the `java` task:

```
<target name="run" depends="jar">

    <java classname="dna.Sequence">

        <arg value="GATTACCA"/>

        <classpath>

            <pathelement location="build/classes"/>

        </classpath>

    </java>

</target>
```

We specify:

- `classname`: the name of the class to run. This should have a `main` method.
- `classpath`: locations that together make up the CLASSPATH. Here we just provide the location of our compiled files.
- `arg`: a command-line argument.

If we run this target:

```
$ ant run

...

run:

     [java] 1909.6000000000001

...
```

This is equivalent to running:

```
$ java -classpath build/lib/dna.jar dna.Sequence GATTACCA

1909.6000000000001
```

7 Provide properties via the command-line

Running Java code in this way, as we have defined it, is inflexible. If we want to run the code on a different DNA sequence e.g. ACGT, then we have to edit the build file. We can use the value of a property instead, by replacing:

```
<arg value="GATTACCA"/>
```

with:

```
<arg value="${dna.sequence}"/>
```

We can now provide a value for this property via the command-line, using ANT's `-DPROPERTY=VALUE` flag. For example:

```
$ ant -Ddna.sequence=ACGT run

...

run:

     [java] 1053.8

...

$ ant -Ddna.sequence=GATTACCA run

...

run:

     [java] 1909.6000000000001

...
```

8 Run tests

Assume we have another directory, `test`, which has a file, `test/dna/SequenceTest.java` that defines a unit test class `dna.SequenceTest`, written using the Junit test framework. Assume we also have the `junit-4.10.jar` file needed to compile the JUnit tests.

Let's write a target to compile our tests.

First we add a new action to our `init` target, to create a directory for our compiled test code:

```
<mkdir dir="build/test-classes"/>
```

We could just put the compiled test code in `build/classes` but it can be useful to keep test code separate, especially when creating JARs where we typically want our test code a separate JARs. We can now define a `compile-tests` target, similar to our `compile` target earlier:

```
<target name="compile-tests" depends="compile">

    <javac srcdir="test" destdir="build/test/classes">

        <include name="**/*.java"/>

        <classpath>

            <pathelement location="build/classes"/>

                <fileset dir=".">

                    <include name="*.jar"/>

                </fileset>

        </classpath>

    </javac>

</target>
```

There are two significant differences between this target and `compile`. We now have a dependency on `compile` to ensure that the code being tested is compiled and up-to-date, before the test code is compiled. We also have a `classpath` element, since to compile the test code we need the path to our compiled code (`build/classes`) and also a JUnit JAR file (the `fileset` here picks up all `.jar` files in the current directory, which includes our `junit-4.10.jar`).

We can now write a target to run our tests. ANT provides a `junit` task specifically for finding, running and reporting on Junit tests. So, we can write our target as:

```
<target name="test" description="Run tests" depends="compile-tests">

    <mkdir dir="build/test/xml"/>

    <junit printsummary="yes"

           haltonfailure="no">

        <classpath>

            <pathelement location="build/classes"/>

            <pathelement location="build/test/classes"/>

            <fileset dir=".">

                <include name="*.jar"/>
```

```
            </fileset>

        </classpath>

        <formatter type="xml"/>

        <batchtest fork="yes"

                   todir="build/test/xml">

            <fileset dir="test">

                <include name="**/*Test.java"/>

            </fileset>

        </batchtest>

    </junit>

</target>
```

The first action creates a `build/test/xml` directory to hold XML documents with our test results.

The key points of the `junit` task are (for full details, see https://ant.apache.org/manual/Tasks/junit.html):

- `printsummary`: should a summary of the test results be printed on standard output. Here, we have requested that they are.
- `haltonfailure`: should the test run stop at the first failure. Here we want all tests to run.
- `classpath`: locations that together make up the CLASSPATH.
- `formatter`: output test reports in XML using the `xml` formatter.
- `batchtest`: run the tests specified within this element, here defined within the `fileset`.
- `fork`: should the tests be run in a separate Java Virtual Machine.
- `todir`: directory in which to put test reports.
- `fileset`: the tests to run. All test files with the suffix `.java` will be found within `test` and its descendants. The associated compiled test classes will then be run.

If we now run this target, we run our tests:

```
$ ant test

Buildfile: build.xml



test:

    [mkdir] Created dir: build/test/xml

    [junit] Running dna.SequenceTest

    [junit] Tests run: 3, Failures: 0, Errors: 0, Time elapsed: 0.022 sec
```

The test reports can be seen in `build/test/xml`. These are in XML. ANT provides a `junitreport` task to merge individual test reports into a merged document, and then transform these into a set of test reports in HTML. Let us add a target to run this task:

```
<target name="test-report">

    <mkdir dir="build/test/html"/>

    <junitreport todir="build/test/xml">

        <fileset dir="build/test/xml">

            <include name="TEST-*.xml"/>

        </fileset>

        <report todir="build/test/html"/>

    </junitreport>

</target>
```

The key points of the `junitreport` task are (for full details, see https://ant.apache.org/manual/Tasks/junitreport.html):

- `todir` in `junitreport`: directory in which to put merged XML reports.
- `fileset`: location and names of individual XML reports.
- `report`: generate the HTML report from the merged documents.
- `todir` in `report`: directory in which to put HTML reports.

HTML test reports can be published online, for example as part of a nightly build-and-test job, to serve as a shared resource for developers on a project to understand the current state of their software.

9 Define a default target

When we created our build file we specified that the default target was `all`:

```
<project name="dna" default="all" basedir=".">
```

So, we should write this target:

```
<target name="all" depends="clean, test, test-report"/>
```

Now we can clean up, then recompile and run our tests and create our HTML test reports as follows:

```
$ ant

$ ls build/test/html/index.html
```

10 Define CLASSPATHs

Our targets `run`, `compile-tests` and `test` all need a CLASSPATH. Though `compile` does not, it might in future if we were to extend our code to need third-party JAR files, for example. This can mean our targets grow, and there is a lot of duplication amongst our CLASSPATHs.

ANT allows us to define paths, including CLASSPATHs, outside of targets, so they can be shared by all targets. For example, we can define the following `path`, with the name `classpath`, which represents a CLASSPATH formed by the locations of our compiled code, compiled test code and JAR files:

```
<path id="classpath">

    <pathelement location="build/classes"/>

    <pathelement location="build/test/classes"/>

    <fileset dir=".">

        <include name="*.jar"/>

    </fileset>

</path>
```

This builds a CLASSPATH that holds (in this order):

- Any compiled Java code in `build/classes` and its descendants.
- Any compiled Java code in `build/test/classes` and its descendants.
- Any files with the suffix `.jar` in the current directory.

ANT's implementation of the `path` construct takes care of any operating system specific issues such as path separators or whether forward or backslashes should be used. Forward slashes are used in ANT files by convention – if running a build file on Windows/DOS, ANT will take care of replacing these with backslashes. We can then update the targets that use a CLASSPATH to refer to this `path`. For example:

```
<target name="compile" depends="init">

    <javac srcdir="src"

           destdir="build/classes" classpathref="classpath">

        <include name="**/*.java"/>

    </javac>

</target>



<target name="run" depends="jar">

    <java classname="dna.Sequence" classpathref="classpath">
```

```
        <arg value="${dna.sequence}"/>
    </java>
</target>

<target name="compile-tests" depends="compile">
    <javac srcdir="test" destdir="build/test/classes">
        <include name="**/*.java"/>
          <classpath>
              <path refid="classpath"/>
          </classpath>
    </javac>
</target>

<target name="test" depends="compile-tests">
    <mkdir dir="build/test/xml"/>
    <junit printsummary="yes"
           haltonfailure="no">
        <classpath>
            <path refid="classpath"/>
        </classpath>
        <formatter type="xml"/>
        <batchtest fork="yes"
                   todir="build/test/xml">
            <fileset dir="test">
                <include name="**/*Test.java"/>
            </fileset>
        </batchtest>
```

```
    </junit>

</target>
```

11 Provide help

We can add a `description` element to our build file summarising what it does e.g.:

```
<description>

  ANT build file to build and test dna package.

</description>
```

We can also add `description` attributes to each `target` describing what each does e.g.:

```
<target name="init" description="Make build directories">
```

If we now run ANT with its `-p` flag, this information will be presented in addition to the targets.

```
$ ant -p

Buildfile: build.xml



  ANT build file to build and test dna package.



Main targets:



 all             Clean, compile, JAR and run tests

 clean           Remove build directories

 compile         Compile code

 compile-tests   Compile tests

 init            Make build directories

 test            Run tests

 test-report     Convert ANT junit XML reports into HTML

Default target: all
```

12 Miscellaneous

12.1 Comments

Build files can be commented:

```
<!-- This is a comment -->

<!--



This is a comment that

spans multiple lines.



-->
```

12.2 Pattern matching

ANT supports pattern matching, which can be useful when specifying file or directory names:

- `?` matches any single character.
- `*` matches zero or more characters.
- `**` matches zero or more directories.

12.3 Shell environment

The shell environment variables can be imported as follows:

```
<property environment="env"/>
```

Their values can be dereferenced using “${env.NAME}“ e.g.

```
<property name="catalina.home" value="${env.CATALINA_HOME}"/>
```

12.4 Inheritance

ANT files can be broken up into multiple files, each with specific, well-defined roles. An ANT file can be imported as follows:

```
<import file="build-root.xml"/>
```

13 Find out more…

Apache provide a Hello World with Apache Ant (https://ant.apache.org/manual/tutorial-HelloWorldWithAnt.html) tutorial.

The Software Sustainability Institute's build and test examples, (https://github.com/softwaresaved/build_and_test_examples) includes sample ANT build scripts for Java.